

Lab 2: Deadlock Avoidance Using Banker's Algorithm

Theory

A deadlock is a situation in an operating system where a group of processes becomes permanently blocked because each process is waiting for a resource that is held by another process in the group. As a result, none of the processes can continue execution, causing the system to halt indefinitely unless corrective measures are taken.

Necessary Conditions for Deadlock

A deadlock can occur only if the following four conditions hold simultaneously:

- **Mutual Exclusion:** At least one resource is non-shareable and can be used by only one process at a time.
- **Hold and Wait:** A process holding one or more resources requests additional resources that are currently allocated to other processes.
- **No Preemption:** Resources cannot be forcibly taken away from a process; they must be released voluntarily.
- **Circular Wait:** A circular chain of processes exists where each process is waiting for a resource held by the next process in the chain.

Methods of Handling Deadlocks

Operating systems use one or more of the following strategies:

- **Deadlock Prevention:** Ensures that at least one of the necessary conditions for deadlock cannot occur.
- **Deadlock Avoidance:** Allocates resources only if doing so keeps the system in a safe state.
- **Deadlock Detection:** Allows deadlocks to occur and periodically checks for their presence.
- **Deadlock Recovery:** Terminates processes or preempts resources to recover from a detected deadlock.

Banker's Algorithm

The Banker's Algorithm, proposed by Edsger W. Dijkstra, is a deadlock avoidance algorithm used to determine whether granting a resource request will keep the system in a safe state. The algorithm checks the availability of resources before allocation to ensure that all processes can complete without causing a deadlock.

Data Structures Used

The algorithm maintains the following data structures:

- **Allocation Matrix:** Number of resources of each type currently allocated to each process.
- **Maximum Matrix:** Maximum number of resources of each type that each process may require.
- **Available Vector:** Number of available instances of each resource type.
- **Need Matrix:** Remaining resources required by each process, calculated as:

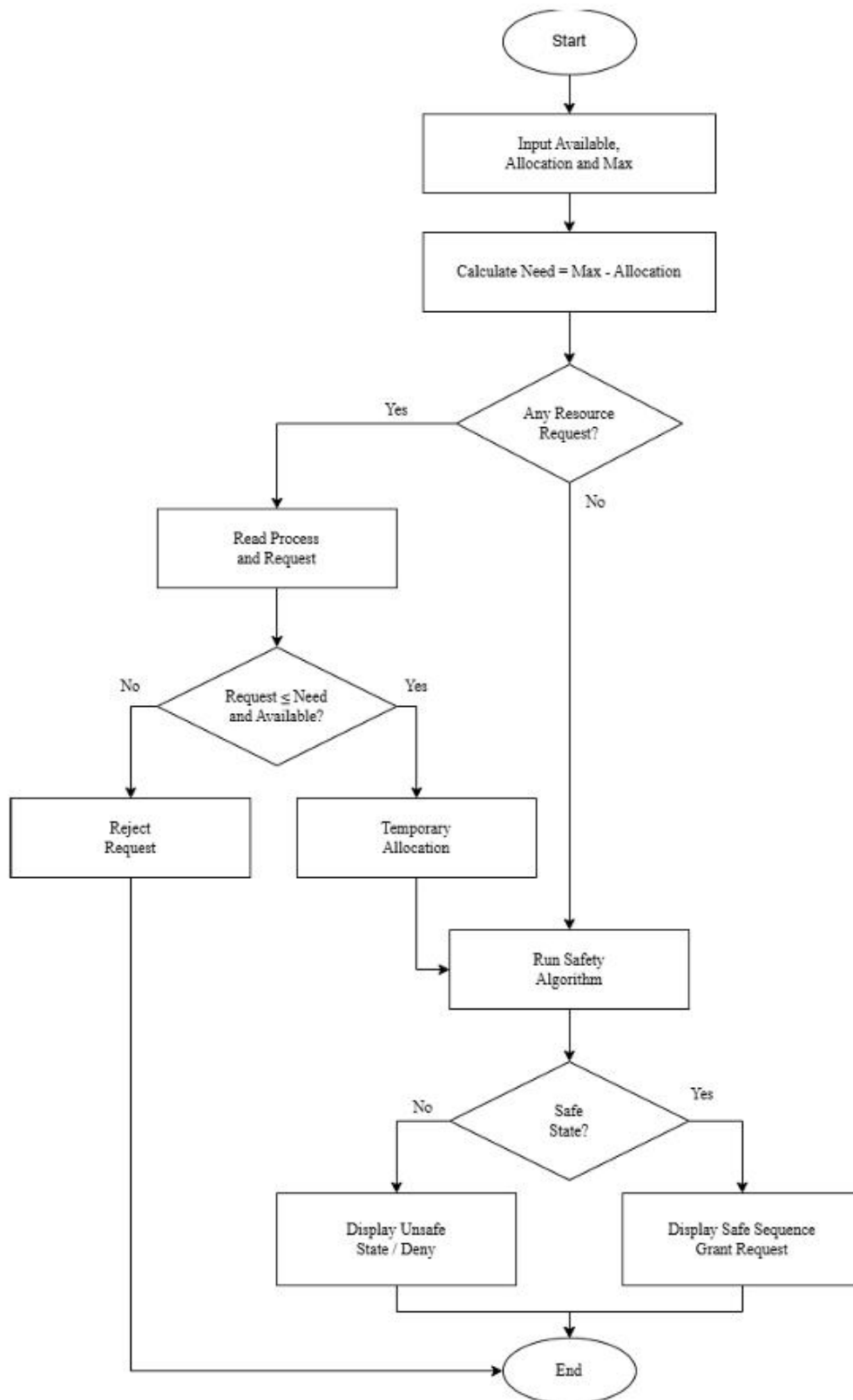
$$\text{Need} = \text{Maximum} - \text{Allocation}$$

Working of the Algorithm

The Banker's Algorithm performs the following steps:

1. Calculate the Need Matrix for all processes.
2. Compare the Need of each unfinished process with the Available resources.
3. If a process's needs can be satisfied, assume it completes execution and releases its allocated resources.
4. Update the Available resources and repeat the process for the remaining processes.
5. If all processes can finish successfully, the system is in a safe state and a safe sequence exists.
6. If no process can proceed at any stage, the system is in an unsafe state, and granting the resource request may eventually lead to a deadlock.

Flow Chart



Advantages

- Prevents deadlocks before they occur.
- Ensures safe allocation of resources.
- Determines whether a safe execution sequence exists.
- Suitable for systems where the maximum resource requirement of each process is known in advance.

Limitation

- Requires prior knowledge of the maximum resource requirements of every process.
- May reduce resource utilization because some requests are denied to maintain system safety.
- Becomes computationally expensive as the number of processes and resources increases.

Source Code:

```
allocation = [  
    [0, 1, 0],  
    [2, 0, 0],  
    [3, 0, 2],  
    [2, 1, 1],  
    [0, 0, 2],  
]
```

```
maximum = [  
    [7, 5, 3],  
    [3, 2, 2],  
    [9, 0, 2],  
    [2, 2, 2],  
    [4, 3, 3],  
]
```

```
available = [3, 3, 2]
```

```
def bankers_algorithm(allocation, maximum, available):  
    process_count = len(allocation)  
    resource_count = len(available)  
  
    need = [  
        [maximum[i][j] - allocation[i][j] for j in range(resource_count)]  
        for i in range(process_count)  
    ]  
  
    work = available.copy()  
    finished = [False] * process_count  
    safe_sequence = []  
  
    while len(safe_sequence) < process_count:
```

```

found_process = False

for i in range(process_count):
    if not finished[i] and all(
        need[i][j] <= work[j] for j in range(resource_count)
    ):
        for j in range(resource_count):
            work[j] += allocation[i][j]

        finished[i] = True
        safe_sequence.append(f"P {i}")
        found_process = True

if not found_process:
    return False, []

return True, safe_sequence

is_safe, sequence = bankers_algorithm(allocation, maximum, available)

if is_safe:
    print("System is in a safe state.")
    print("Safe sequence:", " -> ".join(sequence))
else:
    print("System is not in a safe state.")

```

Output

Process	Allocation (A, B, C)	Maximum (A, B, C)	Available (A, B, C)
P0	[0, 1, 0]	[7, 5, 3]	[3, 3, 2]
P1	[2, 0, 0]	[3, 2, 2]	
P2	[3, 0, 2]	[9, 0, 2]	
P3	[2, 1, 1]	[2, 2, 2]	
P4	[0, 0, 2]	[4, 3, 3]	

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS
System is in a safe state.
Safe sequence: P1 -> P3 -> P4 -> P0 -> P2
○ (base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works>
```

When available = [0, 0, 0]:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS
System is not in a safe state.
○ (base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works>
```

Conclusion

In this lab, Banker's Algorithm was implemented to avoid deadlock by checking whether the system remains in a safe state before allocating resources. Using the allocation, maximum demand, and available resource values, the program calculated a safe sequence of processes. This demonstrates how proper resource allocation can prevent deadlocks and ensure that all processes complete successfully.